Lesson 04: Terraform AWS Provider with LocalStack

Learning Objectives

- Understand what a Terraform provider is
- Configure the AWS provider to connect to LocalStack
- Create your first AWS resource via Terraform (S3 bucket)
- Understand skip credentials validation flags
- Verify resources created by Terraform in LocalStack

---

1. What is a Terraform Provider?

A provider is a plugin that Terraform uses to manage resources in a specific platform. The AWS provider allows Terraform to manage AWS services.

```
Terraform Core
      │
      ├── AWS Provider ──► AWS / LocalStack API
      ├── Local Provider ──► Local filesystem
      ├── Azure Provider ──► Azure API
      └── ... other providers
```

Each provider must be:

1. **Declared** in your configuration
2. **Downloaded** via `terraform init`

---

2. Provider Configuration for LocalStack

The AWS provider needs special configuration to work with LocalStack. Create a new directory:

```
mkdir terraform-localstack
cd terraform-localstack
```

Create `provider.tf`:

```
provider "aws" {
  access_key = "test"
  secret_key = "test"
  region     = "us-east-1"

  endpoints {
    s3       = "http://localhost:4566"
```

```
    dynamodb = "http://localhost:4566"
    sqs      = "http://localhost:4566"
    sns      = "http://localhost:4566"
    lambda   = "http://localhost:4566"
    iam      = "http://localhost:4566"
    sts      = "http://localhost:4566"
  }

  skip_credentials_validation = true
  skip_requesting_account_id  = true
  skip_metadata_api_check     = true
  skip_region_validation      = true
  s3_use_path_style           = true
}
```

Configuration Explained

Setting	Purpose
`access_key` / `secret_key`	Dummy credentials — LocalStack accepts any value
`region`	AWS region; `us-east-1` is the LocalStack default
`endpoints`	Maps each AWS service to the LocalStack endpoint
`skip_credentials_validation`	Skips checking if credentials are valid
`skip_requesting_account_id`	Skips calling STS to get account ID
`skip_metadata_api_check`	Skips EC2 metadata endpoint (not available locally)
`skip_region_validation`	Allows any region string
`s3_use_path_style`	sets path style

---

3. First AWS Resource with Terraform

Create `main.tf` in the same directory:

```
resource "aws_s3_bucket" "first" {
  bucket = "terraform-first-bucket"
}
```

Run the Workflow

```
# Initialize — downloads the AWS provider plugin
terraform init
```

Output:

```
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions...
- Installing hashicorp/aws v5.x.x...
Terraform has been successfully initialized!
```

Notice Terraform downloaded the AWS provider plugin. This is stored in `.terraform/` directory.

```
# Format and validate
terraform fmt
terraform validate

# See what will be created
terraform plan
```

Plan output:

```
Terraform will perform the following actions:

  # aws_s3_bucket.first will be created
  + resource "aws_s3_bucket" "first" {
      + bucket                      = "terraform-first-bucket"
      + id                          = (known after apply)
      + tags_all                    = (known after apply)
      + ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

```
# Create the bucket
terraform apply
```

Type `yes`.

```
aws_s3_bucket.first: Creating...
aws_s3_bucket.first: Creation complete after 0s [id=terraform-first-bucket]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Verify

Via CLI:

```
aws --endpoint-url=http://localhost:4566 s3 ls
```

Via Web UI: Open `http://localhost.localstack.cloud:4566` → S3 → you should see `terraform-first-bucket`.

---

4. Inspecting State

```
terraform state list
```

Output: `aws_s3_bucket.first`

```
terraform state show aws_s3_bucket.first
```

Shows all attributes of the bucket including:

- `id`: `terraform-first-bucket`
- `arn`: `arn:aws:s3:::terraform-first-bucket`
- `bucket`: `terraform-first-bucket`
- `region`: `us-east-1`

---

5. Adding More Resources

Add to `main.tf`:

```
resource "aws_s3_bucket" "first" {
  bucket = "terraform-first-bucket"
}

resource "aws_dynamodb_table" "first" {
  name         = "terraform-first-table"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "id"

  attribute {
    name = "id"
    type = "S"
  }
}

resource "aws_sqs_queue" "first" {
  name = "terraform-first-queue"
}
```

Run:

```
terraform plan
terraform apply
```

Verify all three in CLI:

```
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 dynamodb list-tables
aws --endpoint-url=http://localhost:4566 sqs list-queues
```

---

6. Destroy Everything

```
terraform destroy
```

Type `yes`.

Verify:

```
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 dynamodb list-tables
aws --endpoint-url=http://localhost:4566 sqs list-queues
```

All should be empty.

---

7. Key Takeaways

- The AWS provider must be declared and downloaded via `terraform init`
- LocalStack requires `skip_*` flags and explicit `endpoints` configuration in the provider block
- The `endpoints` map tells Terraform where to reach each AWS service
- Terraform communicates with LocalStack through the same API as real AWS
- All Terraform workflow commands (`init`, `plan`, `apply`, `destroy`) work identically to real AWS
- Resources created by Terraform are visible in the LocalStack Web UI and accessible via AWS CLI

example and exercise repo url

https://github.com/eyu-se/terraform-training-material-with-projects

Trainer - Eyuel M.